VELTECH-AMS-MINI

Project report submitted
in partial fulfillment of the requirement
for award of the degree of

Bachelor of Technology
in
Computer Science & Engineering

By

[STUDENT NAME 1] ([REGISTER NO. 1])
[STUDENT NAME 2] ([REGISTER NO. 2])
[STUDENT NAME 3] ([REGISTER NO. 3])
[STUDENT NAME 4] ([REGISTER NO. 4])

10211CS212 - WEB AND MOBILE APPLICATION DEVELOPMENT

SUMMER 2026-2027

Under the guidance of
[FACULTY NAME], [QUALIFICATION]
ASSISTANT PROFESSOR

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
SCHOOL OF COMPUTING

VEL TECH RANGARAJAN DR. SAGUNTHALA R&D INSTITUTE OF SCIENCE AND TECHNOLOGY

(Deemed to be University Estd u/s 3 of UGC Act, 1956)
Accredited by NAAC with A++ Grade
CHENNAI 600 062, TAMILNADU, INDIA

[MONTH, YEAR]

CERTIFICATE

It is certified that the work contained in the project report titled "VELTECH-AMS-MINI" by [STUDENT NAME 1] ([REGISTER NO. 1]), [STUDENT NAME 2] ([REGISTER NO. 2]), [STUDENT NAME 3] ([REGISTER NO. 3]) and [STUDENT NAME 4] ([REGISTER NO. 4]) has been carried out under my supervision and that this work has not been submitted elsewhere for a degree.

Signature of Supervisor
[FACULTY NAME]
ASSISTANT PROFESSOR
Computer Science & Engineering
School of Computing
Vel Tech Rangarajan Dr. Sagunthala R&D
Institute of Science & Technology
[MONTH, YEAR]

Signature of Head/Assistant Head
of the Department

Dr. M. Kavitha / Dr. T. Kujani
Professor & Head / Associate Professor & Assistant Head
Computer Science & Engineering
School of Computing
Vel Tech Rangarajan Dr. Sagunthala R&D
Institute of Science and Technology
[MONTH, YEAR]

Signature of the Dean

Dr. S. P. Chokkalingam
Professor & Dean
School of Computing
Vel Tech Rangarajan Dr. Sagunthala R&D
Institute of Science and Technology
[MONTH, YEAR]

DECLARATION

We declare that this written submission represents our ideas in our own words and, where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented, fabricated or falsified any idea, data, fact or source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from sources which have not been properly cited or from whom proper permission has not been taken when needed.

(Signature)
[STUDENT NAME 1]
Date: / /

(Signature)
[STUDENT NAME 2]
Date: / /

(Signature)
[STUDENT NAME 3]
Date: / /

(Signature)
[STUDENT NAME 4]
Date: / /

APPROVAL SHEET

This project report entitled “VELTECH-AMS-MINI” by [STUDENT NAME 1] ([REGISTER NO. 1]), [STUDENT NAME 2] ([REGISTER NO. 2]), [STUDENT NAME 3] ([REGISTER NO. 3]) and [STUDENT NAME 4] ([REGISTER NO. 4]) is approved for the degree of B.Tech in Computer Science & Engineering.

Examiners **Handling faculty**

[FACULTY NAME], [QUALIFICATION]

Date: / /

Place:

ABSTRACT

veltech-ams-mini is a unified student web application that brings together academic attendance and campus engagement. It combines the attendance functions of Unieve-Attandance-AI with the events and community functions of Unieven-AI into one responsive interface for Vel Tech students. The application presents student identity, current and upcoming classes, subject attendance, academic reports, registered courses and earned credits alongside events, reels, posts, connections and faculty-created forms.

The system uses a React/Vite interface, a Node.js/Express campus service and a private Python/-FastAPI academic service. The academic adapter reads supported authenticated AMS pages while retaining their report headings, semester divisions and source timestamps. University records remain separate from editable social profiles. Overall results use an interactive cumulative CGPA line graph, clearly labelled published or calculated values, and semester-level PDF and Excel exports. Persistent campus sessions, incremental synchronization, bounded login admission and visibility-aware polling improve continuity and reduce unnecessary requests.

The V1.8 verification record reports 607 passing automated tests, supplemented by browser, responsive-layout and Docker integration checks. Earlier authorized AMS reads verified profile, course, attendance, timetable, results and earned-credit structures for one account. Unavailable source sections are identified explicitly. The project provides an extensible student workspace while preserving the boundary between university-owned records and campus-managed activities.

Keywords: Academic Management System, attendance, campus events, student community, React, FastAPI, source-aware synchronization, progressive web application.

LIST OF FIGURES

3.1 **Architecture diagram of veltech-ams-mini** 7

3.2 **Data flow diagram for academics and campus activities** 9

5.1 **Launched student workspace with demonstration data** 17

9.1 **Phone sign-in and home workspace.** Student, event and grade values shown are fictional demonstration records. 26

9.2 **Subject-first attendance overview and class-planning indicators.** Fictional sample marks demonstrate the subject filters, calendar access and export controls. 27

9.3 **Overall results line graph and CGPA/percentage cards.** The displayed 8.40 and 84% are labelled demonstration estimates, not published student results. 28

9.4 **Earned-credit report divisions and source headings.** Fictional fixture rows demonstrate separate course, category and summary sections. 29

9.5 **Event discovery with Vel Tech banner and free/paid filters.** Event cards and availability are demonstration data. 30

9.6 **Phone event discovery and student community.** The bottom navigation and in-context actions use fictional preview content. 31

9.7 **Full timetable with exact-slot room management.** Fictional source slots demonstrate the administrator view. 32

LIST OF TABLES

1.1 Implemented project scope 3

2.1 User groups and authorization requirements 4
2.2 Hardware and operating prerequisites 5
2.3 Software used in the combined application 6

3.1 Application navigation and content groups 8
3.2 Principal data entities and ownership 9

4.1 Recorded V1.8 automated verification 13
4.2 Representative verified project scenarios 14
4.3 Important corrected failure modes 15

6.1 Data-source outcomes and interpretation 19

LIST OF ACRONYMS AND ABBREVIATIONS

AMS	Academic Management System
API	Application Programming Interface
CGPA	Cumulative Grade Point Average
CSRF	Cross-Site Request Forgery
DOM	Document Object Model
GPA	Grade Point Average
HTTPS	Hypertext Transfer Protocol Secure
JWT	JSON Web Token
OTP	One-Time Password
PDF	Portable Document Format
PWA	Progressive Web Application
QR	Quick Response code
REST	Representational State Transfer
TLS	Transport Layer Security
UI	User Interface
XLSX	Office Open XML spreadsheet format

TABLE OF CONTENTS

Page.No

ABSTRACT **iv**

LIST OF FIGURES **v**

LIST OF TABLES **vi**

LIST OF ACRONYMS AND ABBREVIATIONS **vii**

1 INTRODUCTION **1**

1.1 Introduction 1

1.2 Aim of the project 2

1.3 Project Domain 2

1.4 Scope of the Project 2

1.5 Methodology 3

2 REQUIREMENT SPECIFICATION **4**

2.1 User characteristics 4

2.2 Dependencies 5

2.3 Hardware specification 5

2.4 Software specification 5

3 WEBSITE DESIGN **7**

3.1 Sitemap 7

3.2 Design Phase 8

3.2.1 Data Flow Diagram 8

3.3 Front End and Back End Design 10

3.3.1 Home Page 10

3.3.2 Campus Login and Session Restoration 10

3.3.3 Form Validation 10

3.3.4 Parsing AMS Webpages with Python and the DOM 11

3.3.5 Creation of Webserver Using Node.js 11

3.3.6 Design of the Three-Tier Application with MongoDB 11

3.3.7 Design of Reactive Forms in React 11
3.3.8 Routing and Navigation in React 12
3.3.9 Creation of Microservices 12
3.3.10 Deployment of Microservices 12

4 TESTING **13**
4.1 Testing 13
4.1.1 Test Result 13
4.1.2 Test Bugs 15

5 WEBSITE LAUNCH **16**

6 RESULTS AND DISCUSSIONS **18**
6.1 Website performance 18
6.2 Security 19
6.3 Responsiveness and mobile-friendliness 20

7 CONCLUSION AND FUTURE ENHANCEMENTS **21**
7.1 Conclusion 21
7.2 Future Enhancements 22

8 SOURCE CODE **23**

9 SCREENSHOTS **26**

References **33**

Chapter 1

INTRODUCTION

1.1 Introduction

A student's academic day includes checking attendance, finding a classroom, reviewing results and receiving college updates. Campus participation adds a second set of activities: discovering events, registering for programmes, sharing achievements and communicating with student groups. When these tasks are divided between separate applications, students must repeatedly switch context and distinguish current information from old records.

veltech-ams-mini combines the author's two project repositories, Unieven-AI and Unieve-Attandance-AI, into one student-facing application[2, 3]. The resulting V1.8 system presents academic information and campus engagement through a common navigation structure. Its university setting is Vel Tech Rangarajan Dr. Sagunthala R&D Institute of Science and Technology, Avadi, Chennai[4]. The project retains the original UniEven application emblem and displays the credit "Developed by VEL STUDENTS." It is an independent student project; university records originate from the official AMS rather than from the social community.

The primary screen places the current or next class above a compact student-information panel and attendance overview. Academic pages provide subject attendance, timetable, registered courses, source reports and earned-credit details. Events and community pages provide a different interaction model: discovery, posts, reels, likes, comments, registrations and scoped staff management. These functions share an account interface while keeping official academic identity separate from editable social identity.

A central design issue is the reliability of external data. A portal page can be unavailable, an authenticated session can expire and a table layout can change. The project therefore records source, last attempt, last success and retained-data status for individual academic sections. It does not treat successful attendance retrieval as proof that results, fees or documents were also retrieved.

1.2 Aim of the project

The aim is to develop a responsive, unified student workspace that presents supported AMS records clearly and connects them with campus events and student interaction. The main objectives are to:

1. Combine the two existing projects into one React application and coordinated backend services.
2. Show source-backed student information, class timing and subject attendance with clear calendar and detail views.
3. Preserve AMS result and credit divisions while providing filters, an overall line graph and labelled academic calculations.
4. Support event registration, faculty forms and student community interactions with server-enforced permissions.
5. Restore valid sessions after refresh, reduce redundant synchronization and provide explicit recovery from temporary failures.
6. Supply a repeatable Windows/Docker setup, deployment instructions and an installable web experience.

1.3 Project Domain

The project belongs to educational web applications and campus information systems. It combines an academic information companion, an event-management application and a student community platform. The academic component is primarily read-only integration; the campus component manages its own events, posts, forms, notifications and role assignments.

The implementation uses a browser-based progressive web application (PWA). React provides the interface, the campus API uses Node.js/Express and the academic API uses Python/FastAPI. MongoDB stores campus records and academic snapshots. Docker Compose isolates the services. This division allows an academic connector to change without merging university parsing logic into social or event routes.

1.4 Scope of the Project

The implemented scope is summarized in Table 1.1. A feature's presence in the interface does not establish that a particular student's university account supplies its data.

Table 1.1: Implemented project scope

Module	**Delivered behavior**
Student workspace	Source student facts and portrait, current/next class, attendance cards and college notices.
Attendance and timetable	Subject detail routes, date filters, calendar, calculator, exports, Day/Week/Full timetable and scoped calendar changes.
Academic records	Supported courses, results, assessments, earned credits and source availability for examinations, fees and documents.
Events and community	Free/paid event workflows, registrations, tickets, posts, reels, comments, saves, reports and connections.
Staff and settings	Scoped event roles, faculty forms, review/export, notifications, feedback, device downloads and appearance controls.
Delivery	PWA, Windows PowerShell setup, Docker services and Android packaging configuration.

Official course registration submission remains in AMS through an authenticated portal handoff. The application does not invent a university write endpoint. Paid event tickets depend on configured merchant credentials and server-verified payment confirmation. Placement functions require staff-published or supported source information; absent records are shown as developing or unavailable. No supported separate Home of VelTech private API has been configured, and the retired parent portal is not used.

1.5 Methodology

Development follows an integration-and-verification process. First, the two repository feature sets are consolidated and their active interfaces identified. Second, academic adapters translate inspected, labelled AMS tables into owner-bound records while retaining report metadata. Third, frontend components present those records through dedicated routes and reusable controls. Fourth, campus actions are checked against account ownership and staff scope. Finally, automated checks, Docker integration, browser workflows and responsive screenshots verify the delivered behavior[1].

Synchronization is incremental. A successful report can appear while other sections are still processing. A failed section retains its earlier successful snapshot with its timestamp, rather than erasing it or marking it as fresh. Login admission and report admission have separate bounded queues. Client-side polling pauses when the page is hidden and avoids downloading an unchanged overview.

Verification uses three evidence classes: automated regression tests, fictional integration/demo records and earlier authorized read-only AMS observations. These are reported separately in Chapter 4. No private login, CAPTCHA answer, session cookie or student's report is reproduced in this document. The screenshots show the real interface with labelled demonstration data.

Chapter 2

REQUIREMENT SPECIFICATION

2.1 User characteristics

Students need an interface that works on a narrow phone display and presents their immediate academic tasks without requiring technical knowledge. They can review their own supported records, calculate attendance scenarios, inspect academic trends, register for permitted events, participate in the community and submit available faculty forms. They can edit their social profile and recovery email through the permitted workflow; official university identity is not freely editable.

Faculty members need controlled form creation, evidence review, response exports and publication functions appropriate to their assigned scope. Event organizers need event creation, registration review and check-in functions limited to their club or department. College-wide event managers and administrators require broader management while preserving ownership and visibility rules. Table 2.1 distinguishes account functions from authorization boundaries.

Table 2.1: User groups and authorization requirements

User group	**Required access boundary**
Student	Own academic records, permitted events/forms, and authorized social content.
Faculty	Assigned faculty publications/forms and scoped review; event check-in only where policy allows.
Club organizer	Events within both the assigned club and assigned department.
Department coordinator	Department-scoped event/form management.
College event manager	College-scoped event management.
Placement officer	Placement publications and management within the delivered policy.
Administrator	Account/role assignments, college management, calendar policies, exact-slot room updates and app-feedback responses.

A student-selected social department, display name or college-looking email address cannot grant staff privileges. The backend derives access from persisted account roles and assigned academic department. Official student information is private; the community profile has a separate set of editable fields.

2.2 Dependencies

The academic service depends on the official AMS being reachable and on a valid upstream session. University verification must be completed when the university requests it. Signing into AMS in a separate browser does not transfer that authenticated session to the application's private connector.

MongoDB Atlas is the default Windows database. It requires a database user, appropriate database permissions and a network allowlist for the deployment host. Uploaded social media and form evidence require persistent server disk. SMTP or another configured mail provider is required for real password-recovery delivery. Paid events require merchant keys and a public HTTPS webhook; browser payment success alone does not issue a paid ticket.

Browser camera, location and notification permissions are optional, requested when needed and subject to browser support. HTTPS is required for normal deployed camera/PWA behavior. Localhost development is a limited exception. Free database and storage plans have finite quotas[10]; the project does not rely on unlimited free storage.

2.3 Hardware specification

The repository does not contain a measured minimum-hardware benchmark. Table 2.2 therefore records operating prerequisites rather than fabricated processor, RAM or student-device specifications.

Table 2.2: Hardware and operating prerequisites

Component	**Requirement for this project**
Development computer	A Windows computer capable of running Node.js and Docker Desktop with Linux containers.
Deployment host	Persistent disk, network connectivity, TLS reverse proxy and resources sized through deployment-specific testing.
Student device	A current browser on a computer or phone; touch support for mobile interactions.
Optional camera/location	Camera for live evidence/QR scanning and browser geolocation where a permitted event or form requires them.
Storage	Space for Docker images, application media and backups; Atlas or operator-managed MongoDB capacity.
Network	HTTPS access to the application and AMS, plus configured database and provider connections.

2.4 Software specification

The delivered stack is based on the project's locked dependency manifests and technology chart[1]. The template's example Angular, jQuery and MySQL exercises are implemented here through the actual React, Python and MongoDB design instead of being claimed as additional technologies.

Table 2.3: Software used in the combined application

Technology	**Project use**
HTML5, CSS3, JavaScript	Accessible structure, responsive presentation and interaction.
React 19, Vite 8	Component interface, development server and production/PWA build.
Tailwind CSS 4, Bootstrap 5	Utility and grid styling alongside project-owned components.
React Router 7, Axios	Application navigation and HTTP communication.
Node.js 24, Express 5	Campus REST API, accounts, sessions, events and community.
Python 3.12, FastAPI	Private academic API and authenticated portal integration.
HTTPX, Requests	Verified-TLS academic HTTP clients and compatible adapters.
APScheduler	Scheduled/background academic-service tasks.
PyMongo, Mongoose	MongoDB access for academic and campus data respectively.
JWT, HttpOnly cookies	Private API tokens and separate persistent campus browser sessions.
Docker Compose, PowerShell	Service isolation and Windows configuration/startup.
PWA, IndexedDB	Installable web shell and explicit account-scoped download copies.

The functional requirements include source-aware academic pages, event registration, social interaction, staff-created forms, notifications, settings and permitted exports. Quality requirements include phone readability, explicit loading/error states, session continuity, ownership checks, bounded work queues and repeatable setup. Private API responses are excluded from service-worker offline caching. A device download is an explicit copy of a generated or retrieved file, not an automatic copy of every private record.

Chapter 3

WEBSITE DESIGN

3.1 Sitemap

The application uses a common shell with desktop sidebar navigation and phone bottom navigation. Students reach Home, Attendance, Timetable, Academics, Course registration, Events, Placements, Community, Connections, Feedback & forms, Academic calendar, Student details and Settings. Management routes appear according to persisted permissions. Academic result and subject-attendance details are full content views rather than forced modal windows.

Figure 3.1 shows the deployed logical architecture. The browser contacts the campus service. The private academic service is reached through the campus bridge, rather than being exposed as a second public student API. The database stores separate campus and academic records, while media uses persistent server storage.

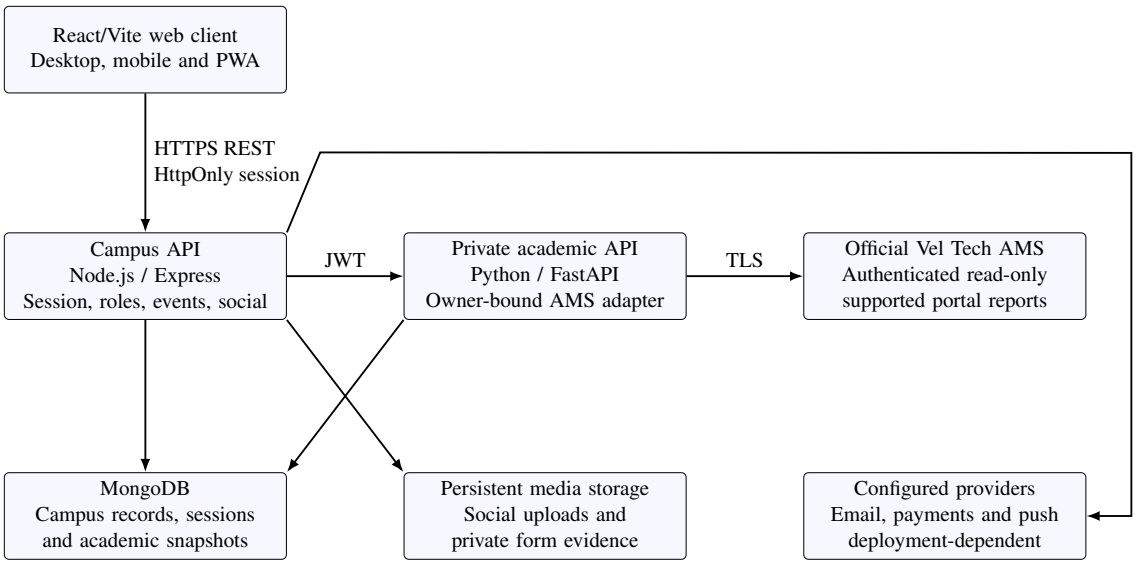


Figure 3.1: **Architecture diagram of veltech-ams-mini**

Table 3.1: Application navigation and content groups

Navigation group	**Principal views**
Daily academics	Home, attendance overview, subject calendar, timetable Day/Week/Full and academic calendar.
Academic records	Registered courses, examinations, results/grades, earned credits, fees/receipts and documents.
Campus participation	Event discovery, event/reel tabs, registration, tickets and staff event management.
Student community	Feed, posts, full-screen reels, profiles, connections, saved/liked/archive views and reports.
Support and controls	Faculty forms, app feedback, notifications, device downloads, appearance and permissions.

3.2 Design Phase

The design separates presentation, campus business logic and academic integration. Official identity, social identity and staff authorization are distinct concepts. A common profile panel may show the student's source name and department, but community edits write only the editable profile fields. The backend never interprets those edits as permission to manage a department.

Academic adapters retain source labels and report divisions. Results remain grouped by source report, semester and examination session; assessment marks are separate from final grades. Earned-credit course rows are separate from category/bucket totals and summary rows. The interface changes their visual presentation without adding unrelated totals together.

3.2.1 Data Flow Diagram

Figure 3.2 shows the main data movement. Identity and scope checks precede private reads or campus writes. Academic snapshots enter storage only after supported parsing; uploaded social media and private form evidence follow separate visibility rules.

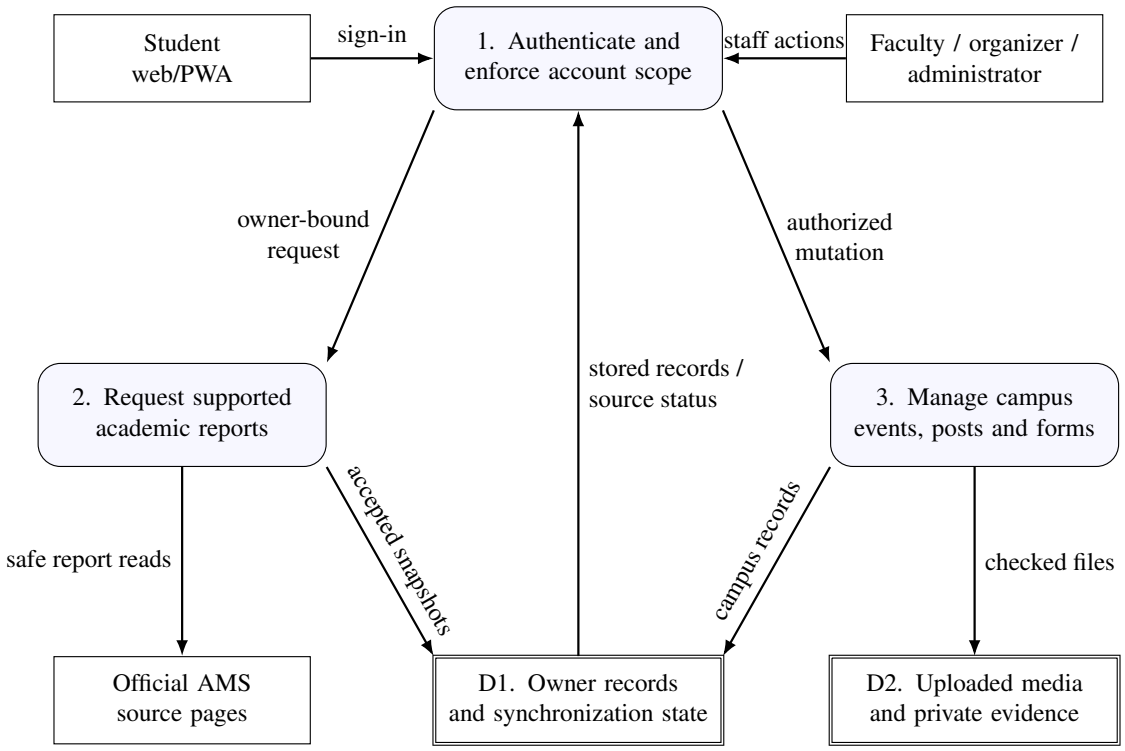


Figure 3.2: **Data flow diagram for academics and campus activities**

The main persistence relationships are shown in Table 3.2. This is a logical summary of the implemented models, not a relational MySQL schema.

Table 3.2: Principal data entities and ownership

Entity group	**Relationship and purpose**
User and CampusSession	A user owns revocable sessions; stored token hashes and expiry support restoration.
CampusAcademicLink	A unique user link stores the academic identity and encrypted bridge token.
Academic records and state	Owner-bound subjects, date marks, timetable, source reports and per-section sync metadata.
Event and EventRegistration	Organizers manage scoped events; each registration belongs to a student and event.
CampusPost	A post/reel belongs to a user and holds visibility, likes, saved users and threaded comments.
Forms, responses and proof	Staff own scoped forms; responses and private proof are linked to the submitting account.
Calendar and room override	Policies/room changes retain the relevant department, date or exact source slot.
Notifications and feedback	Recipient-owned alerts and feedback submissions, with durable retry/deduplication state.

3.3 Front End and Back End Design

3.3.1 Home Page

The home page places the live-class panel at the beginning of the content flow. It uses India time and actual source start/end times to distinguish a current class, the next class and a completed day. It does not fabricate a present or absent state from an aggregate percentage. Missing individual marks stay neutral; dated present and absent marks determine class status.

A student-information panel shows a compact set of source facts, including name, programme/department, semester, year, batch, section and mentor when available. The portrait is shared across the shell and student-details view. Below it, attendance cards summarize overall and subject figures. College notices and optional reels extend the page without replacing its academic priority.

3.3.2 Campus Login and Session Restoration

The student entry screen accepts AMS username and password. A university challenge appears only when AMS requires it; the app does not solve or bypass that challenge. A forgotten-password workflow relates to the configured campus recovery channel and does not change a university password through an undocumented endpoint.

The client begins in a checking state and asks the campus API to restore an existing session. A successful result opens the workspace; an anonymous result opens login; a temporary failure shows retry. Concurrent restores are coalesced, and account-generation guards discard late responses from an earlier account. The rolling 30-day campus cookie and the separate 30-minute, process-local AMS session have different lifetimes. A valid app session therefore survives a refresh even when upstream verification later needs renewal.

3.3.3 Form Validation

Both client and server validate required fields, lengths, identifiers and permitted values. The server enforces account ownership, department scope and available role before mutations. URL handling accepts supported HTTP/HTTPS links and rejects embedded credentials. Evidence uploads use file checks, private ownership and one-use challenges. Attendance and results exports use actual selected records; spreadsheet response exports are formula-safe.

Faculty-created forms can define field types, required responses, submission windows and evidence/location conditions. A live-camera flow binds a fresh challenge to the account and form, expires it and rejects replay. GPS and browser camera checks provide supporting evidence, not a

cryptographic guarantee of physical attendance. Faculty review and scoped ticket scanning remain separate attendance controls.

3.3.4 Parsing AMS Webpages with Python and the DOM

The academic adapter uses verified-TLS HTTP requests and structured HTML parsing. It reads supported labelled rows, inspected attendance grids and safe report-selection forms. Recognized headings determine field meaning. The inspected date/hour attendance layout uses Student Session as its denominator; planned Total Session and Overall % are not substituted as attendance totals.

Unknown tables do not become empty successful reports. Each module records availability, source, attempt and success times. Previously accepted records can be retained after a failed refresh. Safe discovered links and inspected read-only selections are used; feedback, registration and university payment forms are not submitted by the adapter.

3.3.5 Creation of Webserver Using Node.js

The Express campus server exposes REST endpoints for sessions, academic bridging, events, posts, forms, notifications and management. Middleware applies request limits and security headers, excludes private API responses from caching, and rejects unknown endpoints. Media access, CSRF validation and ownership checks accompany the relevant routes. The production frontend can be served from the campus container[9].

3.3.6 Design of the Three-Tier Application with MongoDB

The presentation tier is the React/PWA interface. The application tier contains the campus API and private academic API. The persistence tier contains MongoDB and media storage. Mongoose models support campus actions, while PyMongo supports academic records. Persistent encryption keys protect stored academic bridge tokens; replacing keys during an upgrade can make those stored tokens unreadable.

The campus database stores sessions, users and social/event records. Academic snapshots have their own owner identity and provenance. This separation prevents a public feed query from becoming an academic-record export. Official student-information pages do not offer a bulk identity export.

3.3.7 Design of Reactive Forms in React

Controlled React fields implement event registration, comments, profile editing, calculation rules and faculty forms. Busy, validation, success and retry states are explicit. Paid events wait for verified

server payment state before a ticket is issued. Comment editing is limited to the permitted five-minute window, and replies remain in the active post/reel context.

The project separates immutable academic fields from editable social fields. Username, bio, social links and display preferences belong to the social profile. Recovery-email changes use verification; a derived college email display is not proof that the mailbox has been verified.

3.3.8 Routing and Navigation in React

React Router provides application navigation. On phones, a bottom bar provides the frequent destinations while More opens remaining routes. The layout wraps long titles and contains horizontal timetable/result scrolling within the relevant panel. Dedicated event, ticket and attendance routes preserve browser back behavior. Reels use a full-screen viewing mode; post comments remain contextual.

Optimistic likes and saves provide immediate feedback, then reconcile with the server's authoritative count. Failed writes roll back. Feed ordering uses heuristic recommendations and refresh behavior; the project does not claim a trained Instagram recommendation model. Social activity, academic records and staff management are presented through different permission-aware views.

3.3.9 Creation of Microservices

The campus and academic services have distinct responsibilities and separate health checks. The academic bridge authenticates private service requests and binds them to the owner. Accepted report checkpoints allow progressive loading. Report jobs share three active slots and at most one active job per owner. Cancellation releases its slot.

AMS login uses a different admission controller: four active attempts, eight queued attempts and a five-second queue deadline. Busy requests return HTTP 503 with Retry-After before consuming a challenge. This bounds outstanding work; it is not a guarantee of a university response time.

3.3.10 Deployment of Microservices

Docker Compose defines the private network and services[11]. The delivered topology uses one Docker host, shared database/media and one academic worker with process-local university sessions. Same-host campus replicas share database session/rate-limit state; multi-host academic replication remains future work.

Windows setup uses PowerShell, Node.js 24, Docker Desktop and Atlas. Public deployment adds HTTPS, restricted origins and secure cookies while keeping the academic API private.

Chapter 4

TESTING

4.1 Testing

The verification strategy checks source interpretation, campus permissions and visible user workflows separately. Unit/regression suites cover parsing and calculation rules, event/social behavior, ownership, admission queues and packaging configuration. Integration checks exercise Docker APIs with fictional accounts. Browser checks exercise actual components and responsive navigation. Earlier authorized university reads provide source-layout evidence, without being treated as an automated production test[1].

4.1.1 Test Result

The current verification record, dated 20 October 2026, reports 607 passing automated tests with zero failures or skips. Table 4.1 gives the recorded distribution. These existing project results are reproduced from docs/VERIFICATION.md; preparing this report does not constitute a new live university test.

Table 4.1: Recorded V1.8 automated verification

Test suite	**Passing tests**
React/frontend	163
Campus API	253
Python academic service	180
Android configuration/packaging	11
Total	**607**

The production React/PWA build, rebuilt Docker health checks and fresh-source frozen dependency setup/build also passed in the project record. Android tests validate configuration and packaging tools; they do not establish that an APK was compiled, signed or installed.

Table 4.2: Representative verified project scenarios

ID	Scenario	Verified outcome	Record
T1	Session restore	Valid HttpOnly session survives refresh and simulated PWA update reload; logout revokes session.	Passed
T2	Temporary restoration failure	Retry/online recovery without silently switching to sample records.	Passed
T3	Academic calculation	Blank-field credit joins, source summaries and selected-scale estimates handled separately.	Passed
T4	Result display/export	Overall line graph, semester filters, PDF and OOXML export exercised with fictional records.	Passed
T5	Hidden polling	Zero hidden requests and no overview reload for unchanged synchronization status in the focused test.	Passed
T6	Mobile interface	Login, Events and About checked at 320, 390 and 1440 pixels in light/dark themes.	Passed
T7	Staff permissions	Student/admin boundaries, department scope and exact-slot room changes checked through APIs.	Passed
T8	Notifications/feedback	Retry deduplication, contextual links and silent historical/recovery behavior checked.	Passed
T9	Source work admission	Synthetic login admission and report cancellation respect configured capacities.	Passed

Browser verification included 15 hosted fictional-preview workflows without private API requests or page exceptions. Eighteen branding checks covered three pages, three widths and two themes. Five rebuilt-Docker result workflows covered source-shaped letter grades, credit joins, selected scale persistence, PDF export, summary-only published values and keyboard line-graph selection. These checks use demonstration records, not real university grades.

Earlier authorized reads inspected the actual AMS structures for one account. On 18 October, the recorded read returned 21 profile fields including image metadata, five current courses, 173 re-

sult/assessment rows and 71 earned-credit rows. The result rows comprised 62 final-grade and 111 assessment/internal rows; credits comprised 62 course and nine category rows. Earlier 16 October evidence included 22 timetable entries, five attendance summaries and 284 dated class marks. No unsupported fee ledger or Course Material document data was manufactured when absent.

4.1.2 Test Bugs

The recent fixes address specific failure modes rather than asserting error-free operation under every deployment. Table 4.3 connects observed issues to the corresponding implementation and regression evidence.

Table 4.3: Important corrected failure modes

Issue	**Implemented correction**
Missing overall results	Read explicit result-row/standalone summaries; join source credits and normalize course/semester identifiers.
Incorrect calculation provenance	Published values take precedence; locally selected scales and percentage conversions remain labelled estimates.
Refresh asks to reconnect	Restore campus session before rendering login; preserve an error/retry state for temporary failures.
Old account response arrives late	Abort obsolete requests and verify account generation before accepting responses.
Repeated background downloads	Pause hidden-page polling and fetch overview only after synchronization progress changes.
Phone event/menu collision	Wrap long content, contain panel scrolling and separate toast/More-menu geometry.
Repeated feedback/absence alerts	Persist retry/deduplication keys and suppress historical/recovery alerts.
Unbounded upstream work	Separate bounded login admission from shared report slots and release cancelled reservations.

Actual Windows Desktop operation, live Atlas configuration, physical-phone background push/-sound, camera/GPS evidence, Internet mail, merchant capture/refunds and production failover still need operator validation. The earlier AMS observations establish those inspected account layouts, not every student record or a future unchanged portal layout.

Chapter 5

WEBSITE LAUNCH

The project can run as a React development interface with Docker APIs or as a built application served by the campus container. For the Windows path, the required local tools are Node.js 24 and Docker Desktop using Linux containers. Atlas supplies the database, so no local MongoDB or Ubuntu installation is required.

Initial setup. Extract the source release into one folder. Create an Atlas database user with the required access, add the machine/deployment host to the network allowlist, and obtain the database connection URI. Keep that URI private. From PowerShell in the extracted project folder, run:

```
cd "P:\Intenship\veltech-ams-mini"
powershell -NoProfile -ExecutionPolicy Bypass -File .\Setup.ps1
npm run setup
npm run dev
```

Listing:

Setup accepts the Atlas URI through hidden input, generates persistent private configuration and builds the APIs/frontend. The React development application opens at `http://localhost:5173`; the built Docker application is served at `http://localhost:4100`. The command `npm run doctor` checks the local prerequisites and service/database health. Subsequent API startup uses `Start.ps1`; `Stop.ps1` stops services while retaining persistent data.

Upgrade. Back up the existing private configuration and media. Replace application files while preserving `.private`, environment configuration, encryption keys, database names and Docker volumes. Rebuild/start the services. The campus browser session and stored records are distinct from the temporary AMS session; a private academic-service restart may require fresh university verification.

Public deployment. Set a real HTTPS origin, restrict allowed origins, enable secure cookies and expose only the campus service behind the TLS reverse proxy. Keep the academic API private. Configure SMTP/recovery delivery, database access and backups before student rollout. Payment providers and webhooks must be configured before collecting real event payments. A static preview alone cannot authenticate students or retrieve their private academic data.

Installation and Android. The web build is an installable PWA on supported browsers[12]. The Android packaging guide uses a Trusted Web Activity configuration. The public HTTPS address is

entered as `deploymentUrl` in `android/config.local.json`. Android SDK, private signing identity and Digital Asset Links are additional requirements. The delivered project includes packaging configuration and checks, not a precompiled signed APK.

Figure 5.1 shows the real application's desktop workspace using fictional demonstration records. The values are included to explain the layout; they are not the report authors' academic records.

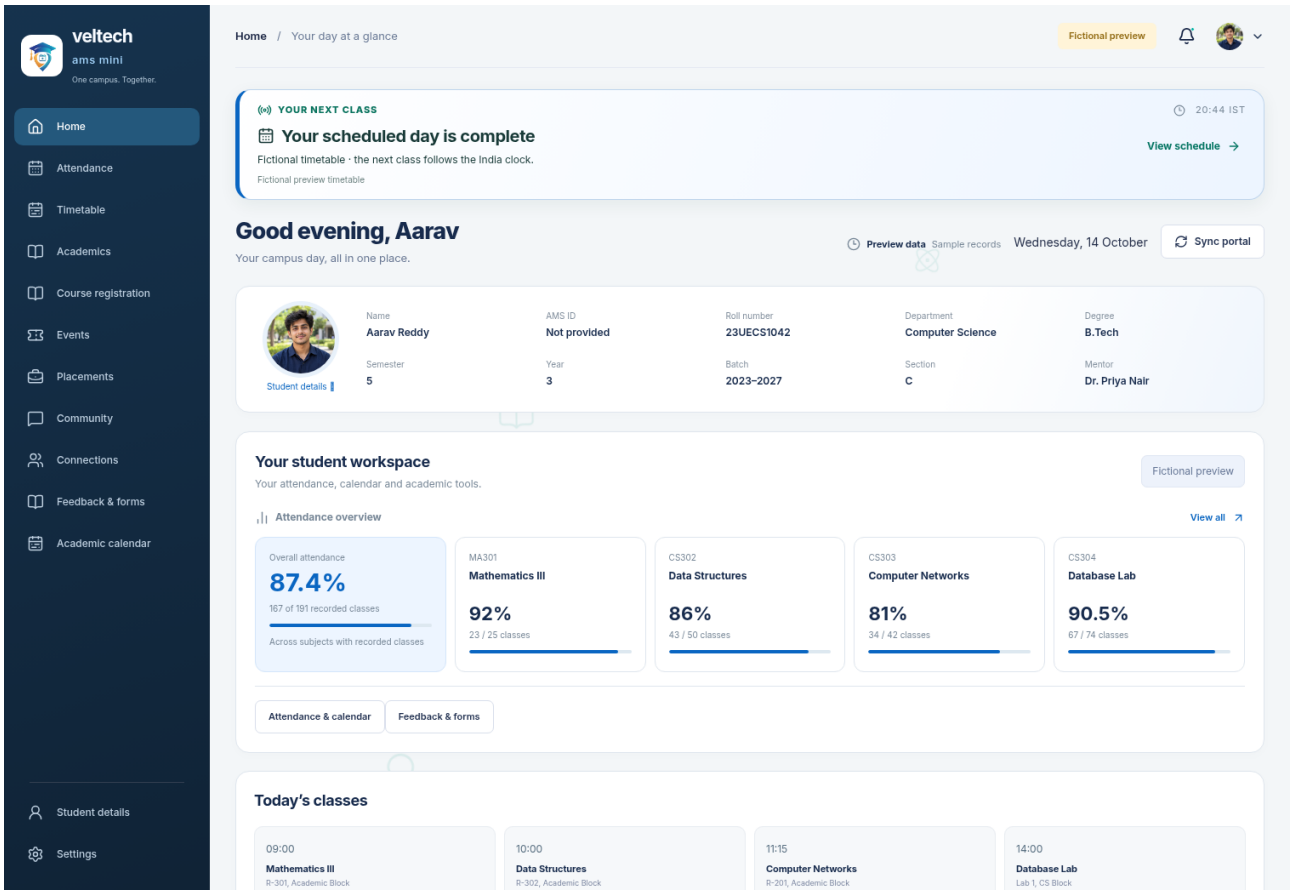


Figure 5.1: **Launched student workspace with demonstration data**

Chapter 6

RESULTS AND DISCUSSIONS

6.1 Website performance

The combined application now exposes the academic and campus modules through one shell. Its primary usability result is direct access to the student's day: current/next class, source student facts and subject attendance. Results and earned credits remain organized by their university report semantics rather than being flattened into unrelated totals.

The top Results panel displays a stock-style line/area graph for cumulative CGPA with a semester-GPA view. Pointer, touch and keyboard selection expose numeric readouts and a crosshair. Overall cards remain independent of the detail filters. Below them, semester, subject, source report, examination session and assessment filters narrow the selected records. Different marking scales are not silently combined into one assessment line.

When source values are available, explicitly published CGPA and percentage take priority. Otherwise, a complete eligible set of course credits and grade points supports a labelled credit-weighted calculation:

$$\mathrm{GPA} = \frac{\sum_{i=1}^{n} c_i g_i}{\sum_{i=1}^{n} c_i}, \qquad \sum_i c_i > 0, \tag{6.1}$$

where c_i is the explicit course credit and g_i is its supported numeric grade point. Cumulative CGPA uses eligible records across the available semesters. Conflicting attempts, missing credits and unsupported grades prevent a complete calculation; source category totals are not course weights. Identical report copies and explicit zero-credit audit rows do not inflate the calculation.

If AMS supplies only letter grades, students can select the applicable visible ten-point scale. These locally mapped values are labelled calculated estimates. The requested default percentage conversion is

$$\text{estimated percentage} = \text{calculated CGPA} \times 10. \tag{6.2}$$

The factor is editable and is not presented as a universal university formula. Published source percentage is used when available. Original grades remain unchanged, and exports distinguish source records from local estimates.

Attendance tools use recorded attended classes P and recorded total classes T, with $T > 0$. Their

basic percentage is $100P/T$. For a chosen target fraction t, where $0 < t < 1$, the calculator uses:

$$\text{additional absences allowed} = \max\left(0, \left\lfloor \frac{P}{t} - T \right\rfloor\right), \tag{6.3}$$

$$\text{consecutive attended classes needed} = \max\left(0, \left\lceil \frac{tT - P}{1 - t} \right\rceil\right). \tag{6.4}$$

These are planning calculations for the entered totals; they do not change official attendance or predict faculty marking.

A recorded local Docker benchmark on 19 October used 24 fictional accounts and eight concurrent student flows. It issued 144 requests with zero failures in 5.0 seconds. Campus password-login latency was 591.0 ms at p50 and 1019.9 ms at p95; private workspace reads were 117.1 ms and 1631.2 ms. This isolated test used no AMS traffic and is not a production capacity or university latency claim.

The latest client checks observed zero hidden-page polling requests and zero overview reloads when sync status had not changed. Bounded source admission limits concurrent university work. These controls reduce redundant activity and fail explicitly under pressure, rather than guaranteeing unlimited simultaneous student logins.

Table 6.1: Data-source outcomes and interpretation

Section	**Evidence and displayed behavior**
Profile, courses, results, credits	Supported structures verified in the earlier authorized account read; report metadata retained.
Attendance and timetable	Earlier dated-mark/source-slot evidence; current display depends on successful owner sync.
Fees, receipts, documents	No supported rows in the inspected account; unavailable state rather than fabricated files/amounts.
Course registration	Supported source courses/window metadata plus official-portal submission handoff.
Home of VelTech	Reference client; no supported independent private API configured.
Placements	Staff/source records when provided; developing/unavailable state otherwise.

6.2 Security

The application uses server-enforced identity, ownership and role checks. Campus browser sessions use an HttpOnly cookie and persisted token hash; logout revokes the stored session. CSRF checks protect state-changing actions. Private academic bridge tokens are encrypted with AES-256-GCM using persistent server configuration. The private service uses JWT authentication. This separates browser sessions from the upstream university session[13].

Community content respects its visibility and recipient scope. Official identity is kept separate from editable social fields. Private form evidence is not a public media attachment. Challenge expiry, one-use tokens, request limits and checked file handling reduce replay and upload risks. Event check-

in requires an authorized actor; location alone cannot prove attendance. Payment tickets rely on verified server state and a configured merchant webhook.

Private academic responses are not stored in the PWA shell cache. Settings Downloads keeps explicit app-generated/retrieved copies in account-scoped IndexedDB: up to 100 files/50 MB, with a 15 MB per-file limit and browser quota constraints. It cannot read files downloaded from other websites or the system Downloads folder. No login secret or actual student report is part of this report's figures or source bundle.

These are implemented controls and tested boundaries, not a security certification. Real-provider configuration, backups, deployment review and monitoring remain necessary before a student-wide release.

6.3 Responsiveness and mobile-friendliness

The interface uses navy, blue and gold accents, readable surfaces and a bottom navigation bar on phones. The live-class panel stays near the top without covering the page during scrolling. Long event names and registration fields wrap within their panels. The timetable grid and wide result tables scroll within their containers instead of expanding the whole page.

Attendance presents subjects first and a calendar beneath, with dedicated subject details and filters. Mobile long-press date inspection supports detailed status review. Reels use full-screen viewing; comments and replies stay in context. Like/save actions provide immediate feedback and reconcile with server state. Settings gives access to appearance, notification/sound choices, permissions, social activity and downloads.

Responsive QA covered widths of 320, 390 and 1440 pixels and both light/dark themes for the latest branding surfaces. The project record reports no page overflow, broken images or page exceptions in those checks. Actual phone installation, operating-system background delivery and physical camera/GPS behavior remain separate device validation tasks.

Chapter 7

CONCLUSION AND FUTURE ENHANCEMENTS

7.1 Conclusion

veltech-ams-mini consolidates two existing college projects into one student workspace. The application combines source-aware academic information with campus events, community interaction, staff forms and notifications. A consistent React interface makes daily information easier to locate while keeping university-owned records separate from campus-managed content.

The academic design preserves the meaning of inspected AMS reports. Attendance uses supplied summaries and dated marks; timetable views use actual source intervals; results retain semester, report and assessment divisions; earned credits retain course/category separation. The overall results graph and numeric cards provide a clearer overview without silently converting missing records into invented scores. Published figures and configurable calculations are labelled separately.

The V1.8 implementation also improves continuity and request behavior. Valid campus sessions restore after refresh, temporary failures offer retry, late account responses are discarded, and unchanged or hidden-page polling avoids unnecessary overview loads. Source login and report jobs use bounded admission. Campus events, community actions and staff management apply persisted permissions and ownership checks.

The recorded 607 passing automated tests, browser workflows and Docker checks support the delivered implementation. Earlier authorized AMS observations support the inspected account layouts. Remaining source/provider dependencies are explicit: unavailable records stay unavailable, official course submission remains in AMS, and real production/provider/device validation is still required. The result is a working project foundation with documented scope rather than a claim that every external service or account is always available.

7.2 Future Enhancements

Future work should first obtain a supported university integration contract or documented API. That would reduce dependence on HTML layouts, provide clearer result/credit schemas and establish whether official course-registration writes may be supported. Additional authorized account/layout fixtures should extend the parser without copying private student values into tests.

Operational enhancements include deployment-specific load tests, monitoring, backup/restore drills and failure recovery. Multi-host deployment requires shared object storage and an explicit strategy for process-local university sessions before academic workers can be replicated safely. Provider validation should cover real email recovery, merchant webhooks/refunds and phone notification behavior.

A signed Android package can be built after the public HTTPS origin, SDK and private signing identity are configured. Physical-device testing should cover installation, camera, QR scanning, location accuracy and browser background restrictions. Accessibility review can further extend keyboard, screen-reader and reduced-motion checks.

Community improvements can add transparent recommendation controls and moderation tools. A trained recommendation model should be considered only with suitable consented interaction data and a measured evaluation; the current heuristic feed should not be described as a trained social-network algorithm. Placement and other university modules should expand only when authorized staff or supported sources can supply reliable information.

Chapter 8

SOURCE CODE

The following excerpts are taken from the current combined project. They illustrate numeric validation, private-token protection and bounded source admission. They are selected implementation examples, not a replacement for the complete application source. No configuration secret or account credential is included.

Academic calculation. The numeric GPA helper accepts explicit non-negative numeric inputs, rejects ambiguous repeated course attempts and exposes whether the available calculation is complete. The caller separately applies a user-selected estimation scale when appropriate, preserving the distinction between source and calculated values.

```
export function academicNumber(value){
 if(typeof value!=='number' && typeof value!=='string')return null;
 if(typeof value==='string' && !/^\d+(?:\.\d+)?$/.test(value.trim()))return null;
 const number=Number(value);return Number.isFinite(number) && number>=0?number:null;
}
export function resultGradePoints(row){return academicNumber(row?.gradePoints ?? row?.gradePoint);}
export function weightedGpa(rows=[]){
 let credits=0,points=0,included=0,excluded=0;
 const attempts=new Map();
 for(const row of rows){const code=row?.courseCode || row?.subjectCode;if(code && resultGradePoints(
     row)!==null)attempts.set(code,(attempts.get(code) || 0)+1);}
 for(const row of rows){const credit=academicNumber(row?.credits),grade=resultGradePoints(row),code=
     row?.courseCode || row?.subjectCode;if(credit===null || credit<=0 || grade===null || (code &&
     attempts.get(code)>1)){excluded++;continue;}credits+=credit;points+=credit*grade;included++;}
 return {value:credits?points/credits:null,credits,included,excluded,complete:included>0 && excluded
     ===0};
```

Listing 8.1: Numeric validation and weighted GPA helpers from the React project

Private token handling. These helpers hash session tokens, verify configured webhook signatures and encrypt private academic link tokens using authenticated encryption. Keys are supplied through persistent private server configuration.

```
import crypto from 'node:crypto';
export const hashToken = token => crypto.createHash('sha256').update(String(token)).digest('hex');
export const newToken = () => crypto.randomBytes(32).toString('hex');
export function checkWebhook(rawBody,signature,secret) {
  if (!Buffer.isBuffer(rawBody) || !secret || typeof signature!=='string' || !/^[a-f0-9]{64}$/i.test(
      signature)) return false;
  const expected=crypto.createHmac('sha256',secret).update(rawBody).digest();
  return crypto.timingSafeEqual(expected,Buffer.from(signature,'hex'));
}
function keyBuffer(key) {
  if (!/^[a-f0-9]{64}$/i.test(key || '')) throw new Error('CAMPUS_LINK_KEY must be a persistent 32-
      byte hex key');
  return Buffer.from(key,'hex');
}
export function encryptToken(token,key) {
  const iv=crypto.randomBytes(12),cipher=crypto.createCipheriv('aes-256-gcm',keyBuffer(key),iv);
  const body=Buffer.concat([cipher.update(token,'utf8'),cipher.final()]);
  return Buffer.concat([iv,cipher.getAuthTag(),body]).toString('base64');
}
export function decryptToken(sealed,key) {
  const bytes=Buffer.from(sealed,'base64');
  const cipher=crypto.createDecipheriv('aes-256-gcm',keyBuffer(key),bytes.subarray(0,12));
  cipher.setAuthTag(bytes.subarray(12,28));
  return Buffer.concat([cipher.update(bytes.subarray(28)),cipher.final()]).toString('utf8');
}
export function fail(status,message) { const error=new Error(message);error.status=status;throw
    error; }
export function httpUrl(value) {
```

Listing 8.2: Campus security helpers: hashes, signature verification and encrypted tokens

Source-login admission. The academic service separates login pressure from record synchronization. The constructor below fixes active/waiting capacity and queue timeout. The complete source file in the LaTeX bundle also contains the asynchronous admission/release flow.

```
            status_code=503,
            detail='AMS sign-in is busy. Please retry shortly.',
            headers={'Retry-After': '3'},
        )


class SourceLoginAdmission:
    """Bound active attempts and their FIFO queue without retaining credentials.

    A task can nest admission when a manual challenge is taken before calling
    the adapter. Child tasks must acquire their own slot, even when they inherit
    the parent's context. Queue cancellation and timeout return reserved slots.
    """

    def __init__(self, max_active: int = 4, max_waiting: int = 8, queue_timeout: float = 5.0):
        if max_active < 1 or max_waiting < 0 or queue_timeout <= 0:
            raise ValueError('Login admission requires positive capacity and timeout.')
        self.max_active = max_active
        self.max_waiting = max_waiting
        self.queue_timeout = queue_timeout
        self._active = 0
        self._waiters: deque[asyncio.Future] = deque()
        self._admitted_task = ContextVar(f'source_login_task_{id(self)}', default=None)

    def _release(self) -> None:
```

Listing 8.3: Academic source-login admission configuration

Application organization. The source release keeps presentation, campus workflows and academic integration in separate folders:

```
veltech-ams-mini/
  frontend/ React/Vite PWA and frontend tests
  services/campus/ Express campus API and tests
  services/academic/ FastAPI AMS adapter and tests
  docker/ Container build configuration
  scripts/ Setup and deployment helpers
  docs/ Verification and operator guides
  android/ Android packaging configuration
  compose.yaml Service/network definitions
  Setup.ps1 Windows private setup
  Start.ps1 Start services
  Stop.ps1 Stop services, retain data
```

Listing:

Chapter 9

SCREENSHOTS

The figures in this chapter show the implemented project interface. All academic grades, names, attendance counts, classroom records, posts and events shown use fictional demonstration or QA fixtures. They explain the product behavior and do not disclose an actual student's records. The Vel Tech photograph identifies the university setting; it is not a synthetic project campus.

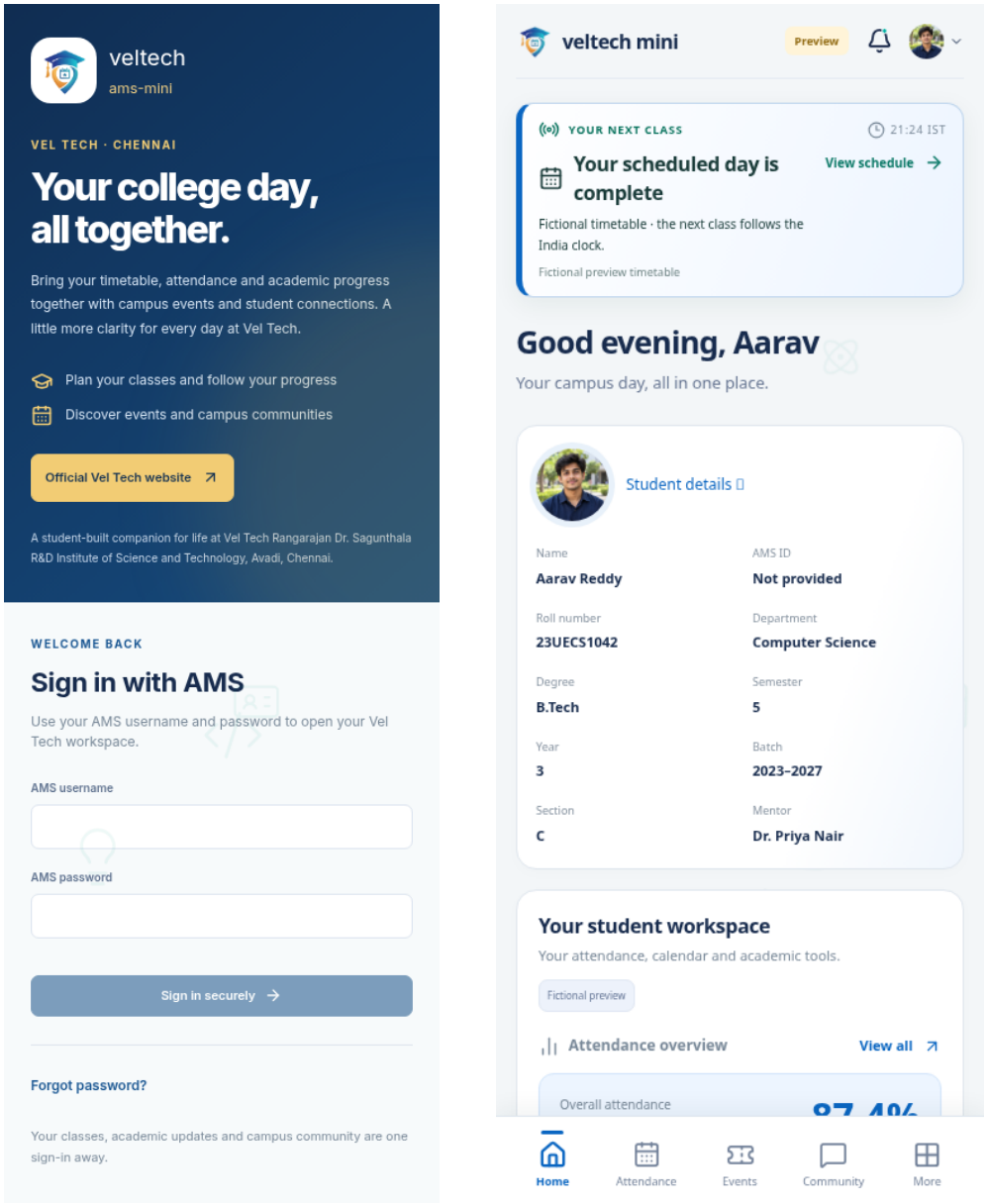


Figure 9.1: **Phone sign-in and home workspace.** Student, event and grade values shown are fictional demonstration records.

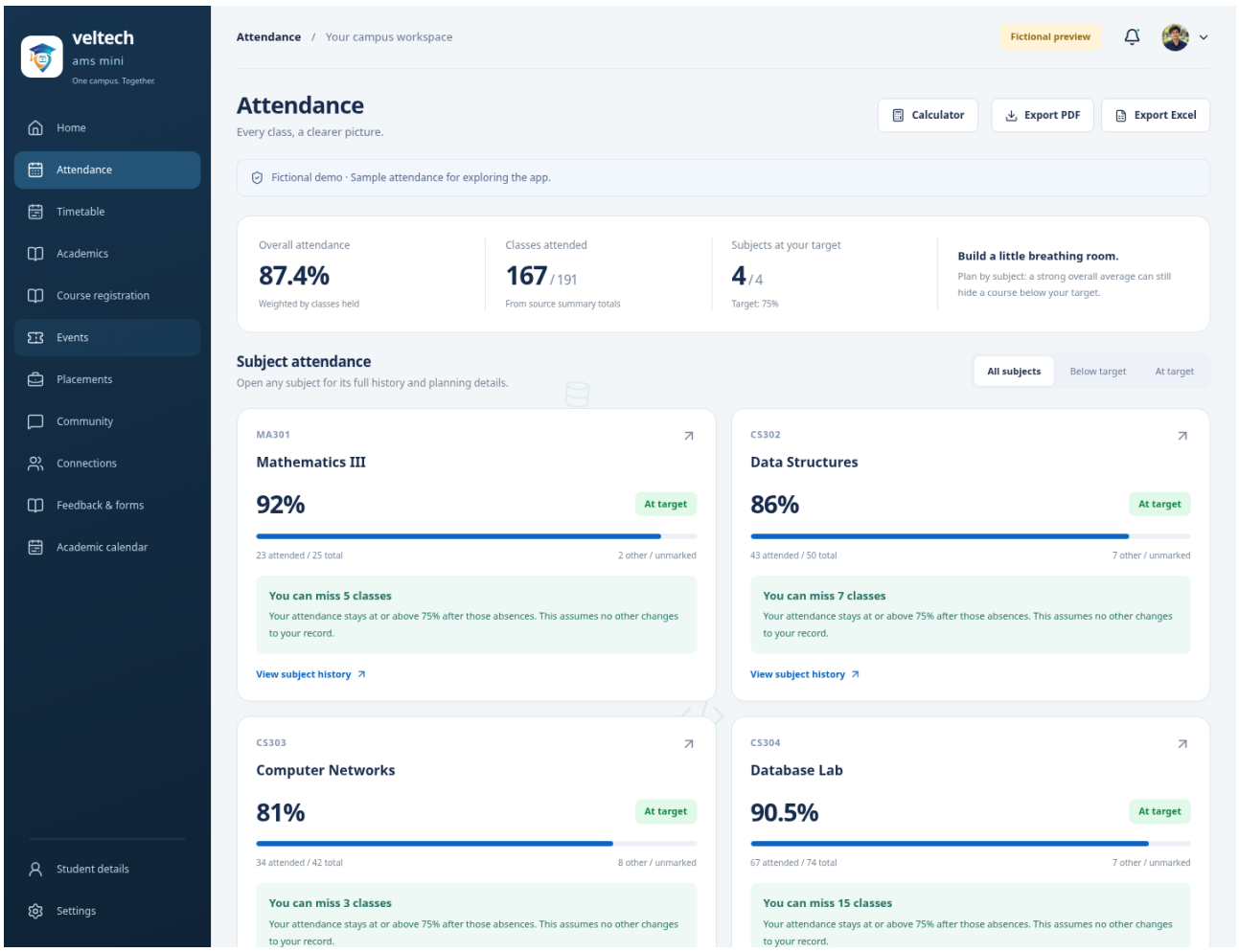


Figure 9.2: **Subject-first attendance overview and class-planning indicators.** Fictional sample marks demonstrate the subject filters, calendar access and export controls.

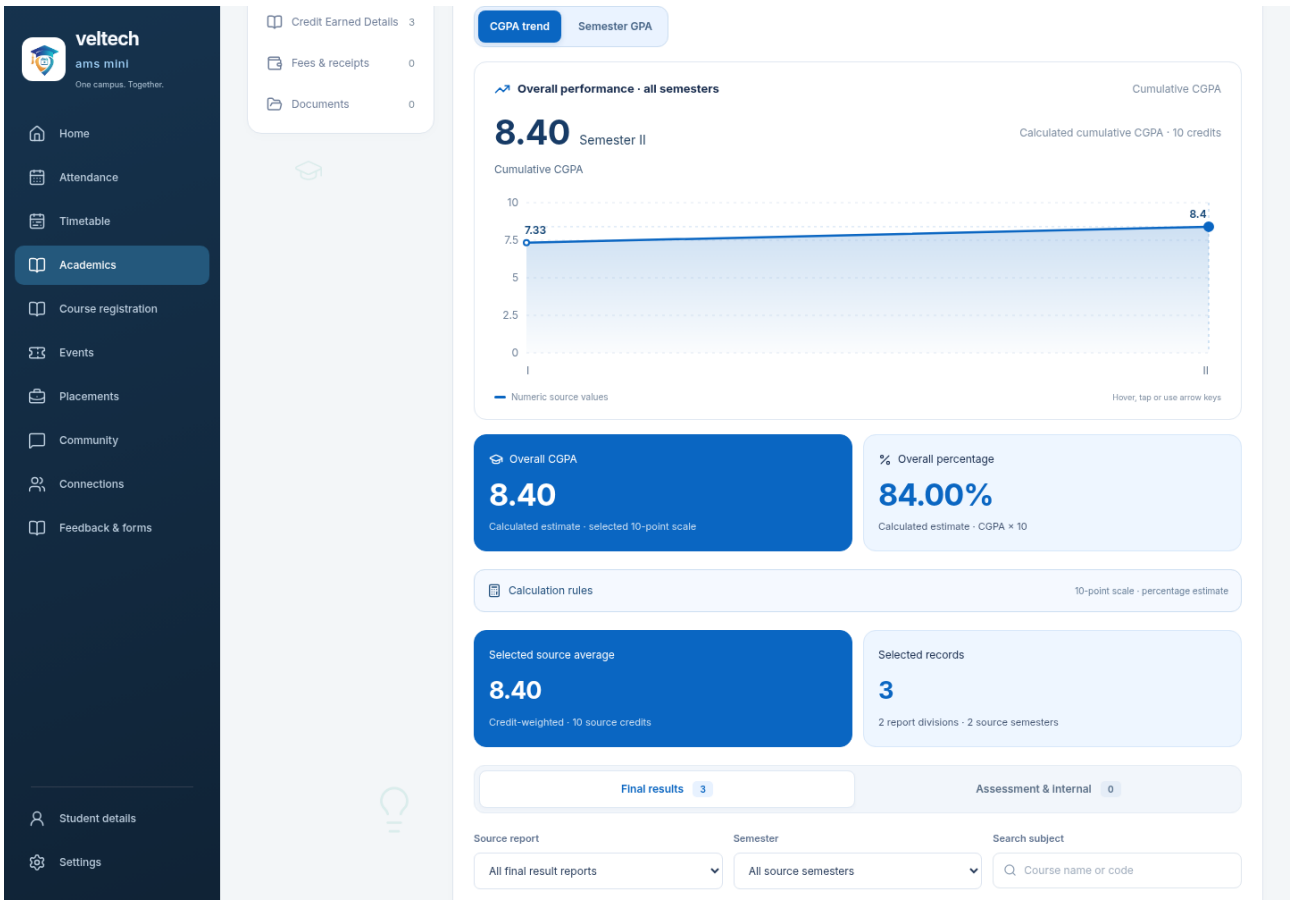


Figure 9.3: **Overall results line graph and CGPA/percentage cards.** The displayed 8.40 and 84% are labelled demonstration estimates, not published student results.

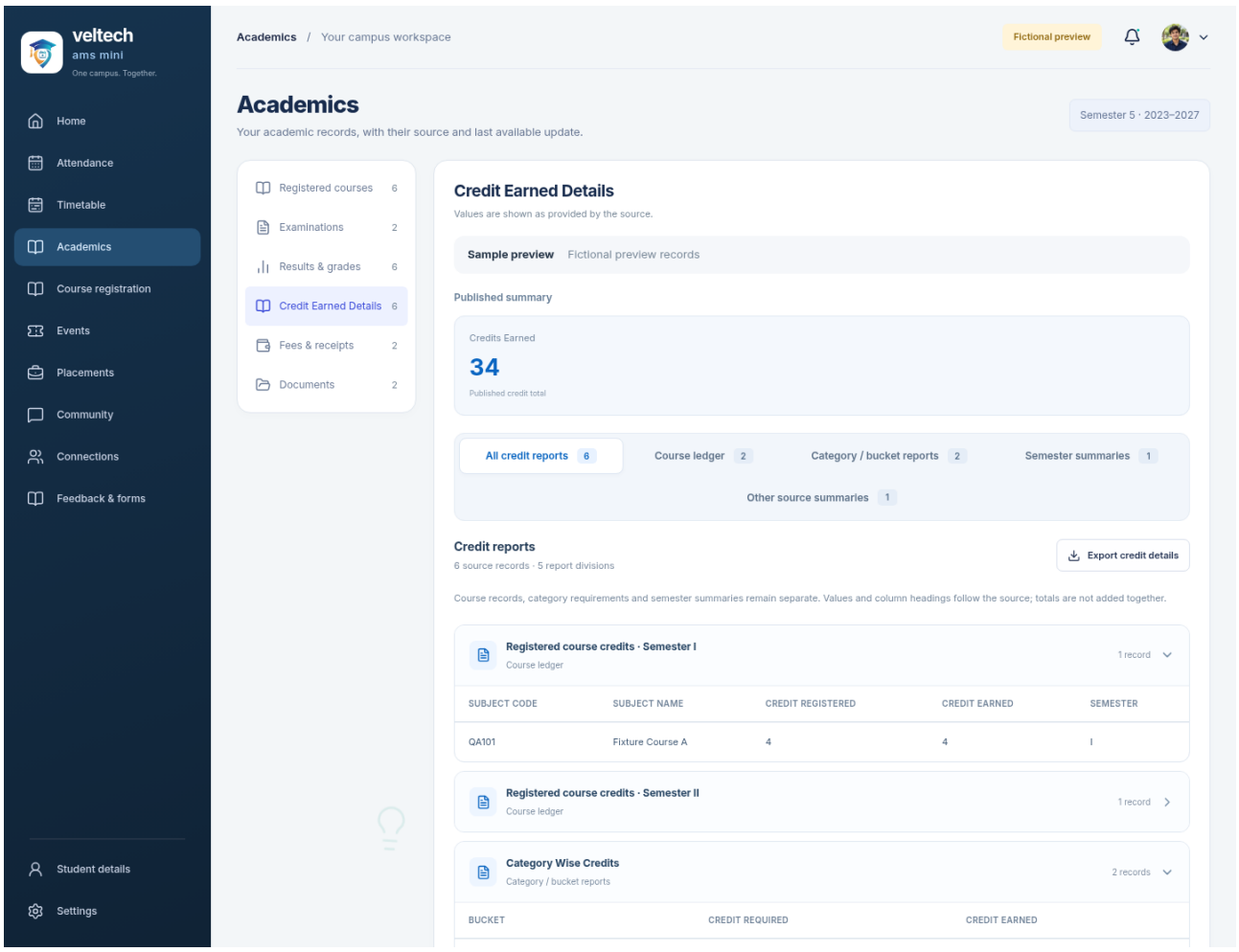


Figure 9.4: **Earned-credit report divisions and source headings.** Fictional fixture rows demonstrate separate course, category and summary sections.

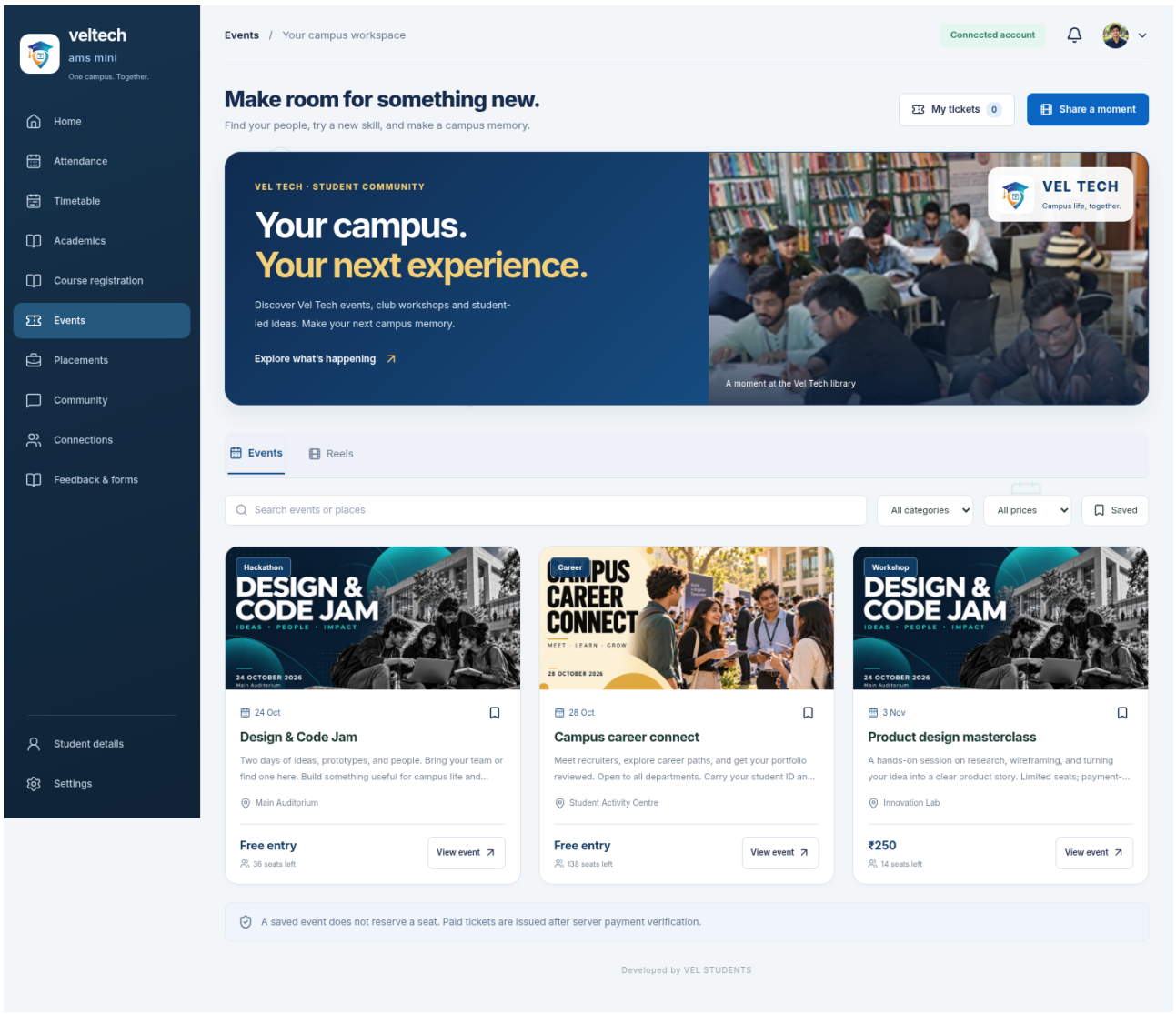


Figure 9.5: **Event discovery with Vel Tech banner and free/paid filters.** Event cards and availability are demonstration data.

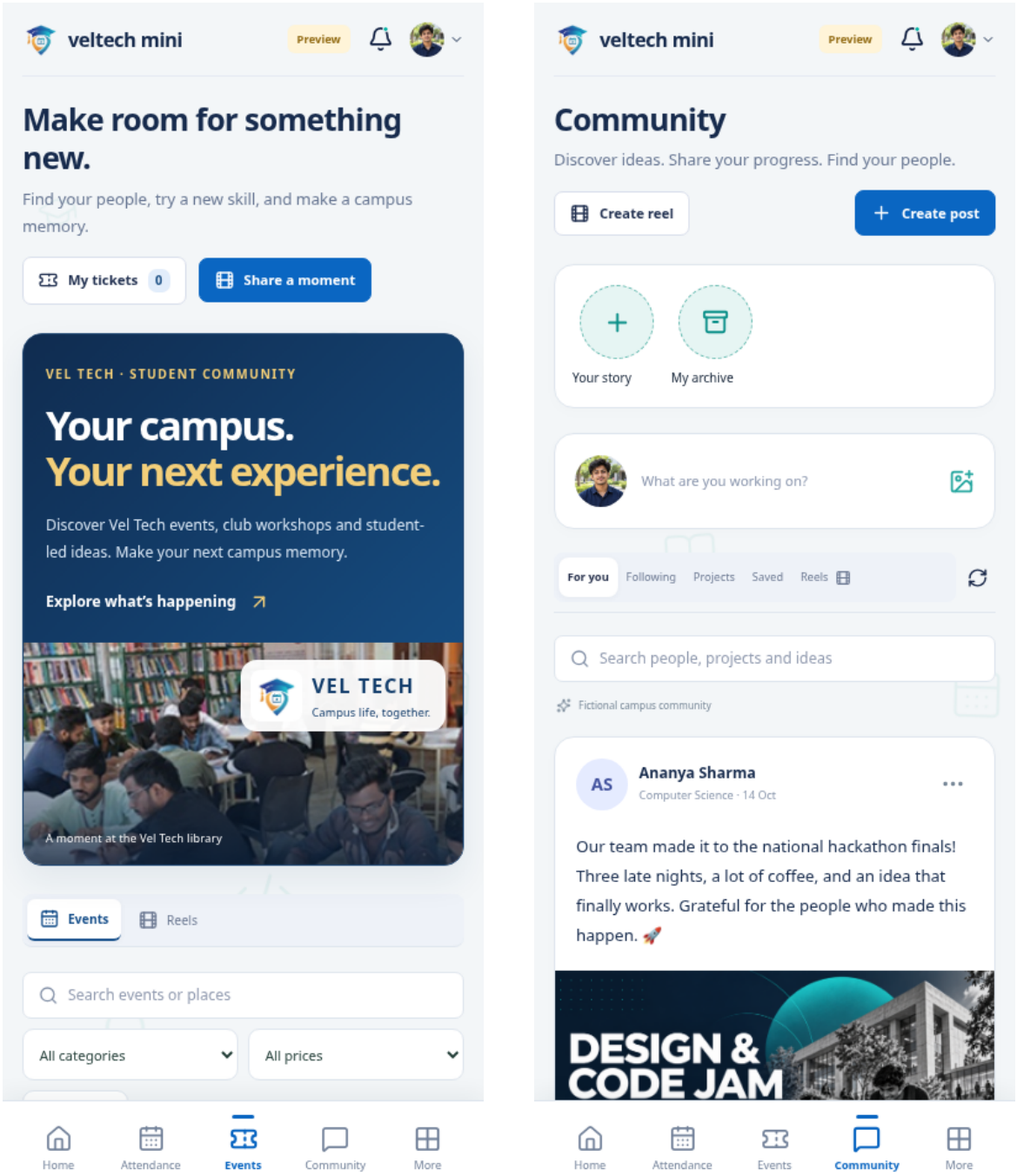


Figure 9.6: **Phone event discovery and student community.** The bottom navigation and in-context actions use fictional preview content.

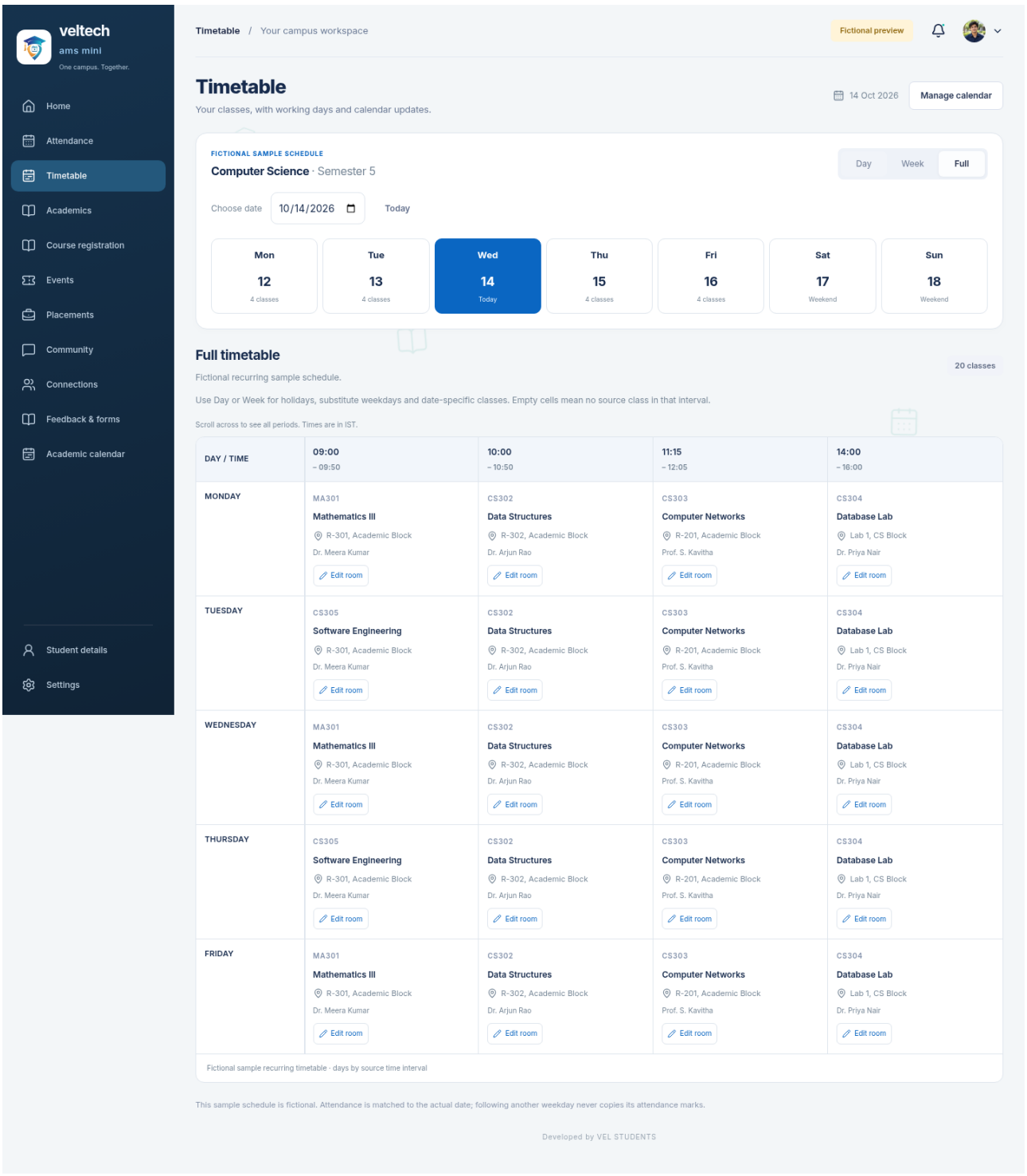


Figure 9.7: **Full timetable with exact-slot room management.** Fictional source slots demonstrate the administrator view.

References

[1] VEL STUDENTS, *veltech-ams-mini V1.8: source code and project documentation*, 20 October 2026. Project files: README.md, docs/FEATURES.md, docs/IMPLEMENTATION.md, docs/VERIFICATION.md and docs/technology.md.

[2] mprem27, *Unieven-AI*, project source repository. `https://github.com/mprem27/Unieven-AI`.

[3] mprem27, *Unieve-Attandance-AI*, project source repository. `https://github.com/mprem27/Unieve-Attandance-AI`.

[4] Vel Tech Rangarajan Dr. Sagunthala R&D Institute of Science and Technology, *Official university website*. `https://www.veltech.edu.in/`.

[5] Vel Tech, *Academic Management System*. `https://ams.veltech.edu.in/Default.aspx`. Credit Earned Details: `https://ams.veltech.edu.in/CreditDetails.aspx`.

[6] React contributors, *React documentation*. `https://react.dev/`.

[7] Vite contributors, *Vite guide*. `https://vite.dev/guide/`.

[8] FastAPI contributors, *FastAPI documentation*. `https://fastapi.tiangolo.com/`.

[9] Express contributors, *Express documentation*. `https://expressjs.com/`.

[10] MongoDB, *Atlas documentation and free-cluster limitations*. `https://www.mongodb.com/docs/atlas/`. `https://www.mongodb.com/docs/atlas/reference/free-shared-limitations/`.

[11] Docker, *Docker Compose documentation*. `https://docs.docker.com/compose/`.

[12] MDN contributors, *Progressive web apps*. `https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps`.

[13] OWASP Foundation, *Session Management Cheat Sheet*. `https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html`.